

HACKATHON 2026 PROJECT

Emergency Response

עזרה ראשונה — A mobile-first, Hebrew (RTL) first-aid training app with a local, offline AI emergency assistant.

React 18 + TypeScript

Vite

Tailwind CSS

FastAPI (Python)

LangChain + Chroma

Ollama (local LLM)

faster-whisper (STT)

Project Summary & Codebase Walkthrough — prepared for hackathon judges

Repository: matanmiller/Hackathon2026 · Generated 2026-06-12

1. What is this project?

Emergency Response (internal package name `ezra-rishona-ai`, Hebrew: עזרה ראשונה — "First Aid") is a mobile-first, single-page web app that teaches everyday people how to react in medical and civil emergencies, and gives them a local, privacy-friendly **AI emergency assistant** they can talk to (by text or voice) when they need step-by-step guidance in the moment.

The entire UI is in **Hebrew, right-to-left (RTL)**, and designed to look and feel like a phone app — a single rounded "device frame" that fills the screen on mobile and centers itself on desktop.

The three tabs

Tab	Hebrew label	Purpose
Learn	למידה	Browse emergency categories → lessons → either a step-by-step illustrated walkthrough or a timed "real scenario" quiz.
Quiz	בוחן מהיר	A short multiple-choice knowledge check covering all lessons, with a final score.
AI Agent Chat	סוכן חירום	A chat interface (text + voice) backed by a local Python AI server that answers first-aid questions from a curated knowledge base.

The "elevator pitch" for judges

- **Real content, not placeholder text** — lesson text and illustrations were extracted directly from real first-aid reference documents (Word/PDF) supplied for the hackathon.
- **Designed like a real consumer app** — RTL Hebrew UI, custom icon set, color-coded categories, animations, a rotating "did you know" medical quote on the home screen, and a pulsing "Call AI Agent" emergency button.
- **Two learning modes** — a guided illustrated walkthrough *and* a timed "what would you do?" scenario quiz with a countdown bar, simulating the pressure of a real emergency.
- **AI runs 100% locally** — no cloud API keys, no internet dependency, no data leaves the device. Uses Ollama for the LLM/embeddings and faster-whisper for speech-to-text, fully offline-capable.
- **Voice-first accessibility** — users can speak their emergency instead of typing, which matters when hands are busy helping a casualty.

2. Technology Stack & File Types

Frontend

Technology	Version	Role
TypeScript	5.4.5	Primary language for all app logic and components (strict typing via <code>src/types.ts</code>)
React	18.3.1	UI framework — function components + hooks only (no class components, no Redux)
Vite	5.3.1	Dev server & build tool — instant HMR, builds to <code>dist/</code>
Tailwind CSS	3.4.4	Utility-first styling, custom theme (colors, fonts, animations)
vite-plugin-mkcert	2.1.0	Generates a locally-trusted HTTPS certificate so the microphone API works over LAN (e.g. testing on a phone)

Backend ("ChatBotBackend")

Technology	Role
Python 3.12	Backend language
FastAPI + Uvicorn	HTTP API server (runs at <code>http://127.0.0.1:8000</code>)
LangChain (community/core/chroma)	Orchestrates the Retrieval-Augmented-Generation (RAG) pipeline
Ollama (<code>OllamaEmbeddings</code> + <code>Ollama</code> LLM)	Local embedding model (<code>nomic-embed-text</code>) and local LLM (<code>qwen2.5:0.5b</code>) — both run entirely on-device
ChromaDB (via <code>langchain-chroma</code>)	In-memory vector store for semantic search over the first-aid knowledge base
faster-whisper	Local speech-to-text (Whisper "tiny" model), transcribes Hebrew voice recordings

File extensions used in this project

Extension	What it's for	Examples
<code>.tsx</code>	React components (TypeScript + JSX)	<code>App.tsx</code> , <code>ChatTab.tsx</code> , <code>LessonScenario.tsx</code>
<code>.ts</code>	Non-visual TypeScript: types, data, services, hooks, config	<code>types.ts</code> , <code>content.ts</code> , <code>geminiService.ts</code>
<code>.css</code>	Global styles + Tailwind directives	<code>src/index.css</code>
<code>.html</code>	App entry HTML / output	<code>index.html</code>
<code>.json</code>	Static data & the AI knowledge base	<code>content.ts</code> 's data sources, <code>quotes.ts</code> data, <code>knowledge_base.json</code>
<code>.py</code>	AI backend server	<code>ChatBotBackend/main.py</code>
<code>.txt</code>	Python dependency list	<code>requirements.txt</code>
<code>.png</code> / <code>.jpeg</code> / <code>.svg</code>	Lesson illustrations, scenario images, app icons	<code>src/assets/lessons/...</code> , <code>icons_images/...</code>

.docx / .pdf	Original source material the lesson content was extracted from	categories/Medical_Conditions/*.docx/.pdf
--	--	---

3. High-Level Architecture

The app is split into two independent processes that talk to each other only over `localhost` HTTP:



Why this matters for judges: the AI assistant works without any internet connection or paid API key — everything (embeddings, vector search, the language model, and speech-to-text) runs on the laptop hosting the demo.

4. Project Structure

```
Hackathon2026/
├── index.html           Vite entry point
├── package.json         Frontend deps & scripts (dev/build/preview)
├── vite.config.ts       Vite config: React plugin + mkcert HTTPS, host:true
├── tailwind.config.js   Custom theme: Heebo font, "emergency" red, animations
├── tsconfig*.json      TypeScript project config (only "src" + vite.config.ts)
├── src/
│   ├── main.tsx         React root render (mounts <App />)
│   ├── App.tsx          App shell: phone-frame layout, tab switcher
│   ├── index.css        Tailwind directives + global styles
│   ├── types.ts         All shared TypeScript types/interfaces
│   └── components/
│       ├── Header.tsx   Top bar with app icon + title
│       ├── BottomNav.tsx 3-tab bottom navigation bar
│       ├── LearnTab.tsx  Orchestrates the whole "Learn" flow (state machine)
│       ├── CategoryCard.tsx Square card for a category (medical/nature/...)
│       ├── LessonSquareCard.tsx Square card for a lesson, shows progress badge
│       ├── LessonChoice.tsx "Lesson or Quiz?" choice screen for a lesson
│       ├── LessonPager.tsx Paginated illustrated lesson walkthrough
│       ├── LessonDetail.tsx Fallback checklist view (lessons w/o pages)
│       ├── LessonQuiz.tsx Standard MCQ quiz for a lesson
│       ├── LessonScenario.tsx Timed "what do you do next?" scenario quiz
│       ├── QuizTab.tsx   Global quick-quiz tab (all lessons)
│       ├── QuizQuestionCard.tsx Single MCQ card with explanation feedback
│       ├── ChatTab.tsx   AI agent chat UI (messages, input, mic, send)
│       ├── ChatBubble.tsx Single chat message bubble (user/agent)
│       ├── MicPermissionPrompt.tsx Inline help text for mic permissions
│       ├── RotatingQuote.tsx Rotating "famous medical quote" widget
│       ├── accentStyles.ts Color theme system (red/sky/amber/violet)
│       ├── icons.tsx / icons.types.ts Hand-rolled SVG icon components
│       ├── categoryIcons.ts Maps category id → icon component
│       └── lessonIcons.ts Maps lesson icon key → icon component
│   ├── data/
│       ├── content.ts    Categories, lessons, lesson pages, scenario quizzes, and quick-quiz questions (Hebrew)
│       ├── quotes.ts     Loads/exports the rotating medical quotes
│       └── famous_medical_quotes.json 100 famous medical quotes (source data)
│   ├── hooks/
│       └── useVoiceRecorder.ts Mic recording state machine (MediaRecorder API)
│   ├── services/
│       ├── geminiService.ts Calls AI backend /api/chat
│       └── transcriptionService.ts Calls AI backend /transcribe
│   └── assets/
│       ├── app-icon.png App header icon
│       └── lessons/choking|bleeding/... Lesson + scenario illustrations
├── ChatBotBackend/
│   ├── main.py          FastAPI app: RAG chat endpoint + Whisper STT endpoint
│   └── knowledge_base.json Curated first-aid facts (the AI's "brain")
```

```
| └─ requirements.txt           Python dependencies
|
| └─ categories/Medical_Conditions/ Source .docx/.pdf reference documents
| └─ icons_images/             Extra raw images used for scenario quizzes
| └─ famous_medical_quotes/    Source JSON for the rotating quotes
```

5. Frontend Walkthrough

5.1 App Shell — App.tsx

The whole app renders inside one `<div dir="rtl" lang="he">` "phone frame": full height on mobile, a centered rounded card (max-width `448px`, fixed height `850px`) with a shadow on desktop. It has three regions, top to bottom:

- **Header.tsx** — app icon + "Emergency Response" title.
- **Main content** — one of **LearnTab**, **QuizTab**, or **ChatTab**, swapped via a single `activeTab` state (`'learn' | 'quiz' | 'chat'`). The chat tab gets a non-scrolling flex layout (its own internal scroll area); the other tabs get a scrollable padded area.
- **BottomNav.tsx** — 3 tab buttons: למידה (Learn), בוחן מהיר (Quiz), סוכן חירום (AI Agent).

5.2 The Learn tab flow — LearnTab.tsx

This component is the heart of the app. It is a small state machine driven by 4 pieces of state: `selectedCategoryId`, `selectedLessonId`, `lessonView` (`'choice' | 'lesson' | 'quiz'`), and `completedSteps`. The screens render in this order:

1. **Home screen** — a big pulsing red "Activate AI Emergency Agent" button (animated with a custom `pulse-strong` keyframe) that jumps straight to the AI Chat tab, a 2×2 grid of **CategoryCards** (Medical Emergency, Natural Disasters, Accidents, War Emergency — each with its own accent color), a disclaimer that this is training only (call 101 in real emergencies), and the **RotatingQuote** widget.
2. **Category screen** — a 2×2 grid of **LessonSquareCards** for the selected category, each showing a progress badge (`completed/total` steps). Categories with no lessons yet show a "coming soon" placeholder.
3. **LessonChoice.tsx** — once a lesson is picked, the user chooses between "Learn the lesson" and "Take the quiz" (only shown if the lesson has quiz/scenario content).
4. **Lesson content** — either:
 - **LessonPager.tsx** — for lessons with a `pages` array (both current lessons): a swipeable/paginated walkthrough where each PDF-derived paragraph is rendered as its own illustrated "card" (image + text), grouped under a page header, with a progress bar and a completion callback that marks all steps done.
 - **LessonDetail.tsx** — fallback for lessons without `pages`: a simple checklist of `lesson.steps` the user can tick off manually.
5. **Quiz content** — either:
 - **LessonScenario.tsx** — for lessons with a `scenario` array (currently the bleeding lesson): a **timed scenario quiz**. Each round shows an image of the situation for a "reveal" phase (4s, with the prompt `מה הפעולה הבאה שיש לבצע?` — "what's the next action?"), then switches to a "choice" phase with 3 possible actions and a **6-second countdown progress bar**; picking the correct one (or running out of time) shows feedback before moving to the next scenario.
 - **LessonQuiz.tsx** — standard multiple-choice quiz for lessons that only define `steps` /quick-quiz questions.

5.3 The Quiz tab — QuizTab.tsx + QuizQuestionCard.tsx

A separate, top-level "quick quiz" (בוחן מהיר) that iterates over `QUIZ_QUESTIONS` from `content.ts` (one question per lesson today: choking and massive bleeding). Each **QuizQuestionCard** lets the user pick an answer and immediately shows a correct/incorrect explanation. Once every question is answered, a summary screen shows the score with a restart button.

5.4 The AI Agent Chat tab — ChatTab.tsx

This is the most technically interesting tab — it combines a chat UI with local voice transcription:

- The chat opens with a canned agent greeting: **מה קורה. אני סוכן החירום שלך. תאר לי את המצב בקצר ובצורה ברורה. מה קורה? כעת?** ("Hello, I'm your emergency agent. Briefly describe the situation. What's happening now?").
- Messages are rendered as **ChatBubbles** (user vs. agent styling), with an animated "typing" indicator (**"הסוכן מקליד..."**) while waiting for a response.
- **Sending a message** calls `sendMessageToAgent()` in `geminiService.ts`, which `POST`s `{ question }` as JSON to `http://127.0.0.1:8000/api/chat` and displays `data.answer`. If the backend is unreachable, it shows a friendly Hebrew error explaining the local AI server / Ollama needs to be running.
- **Voice input** (🎤 button) is powered by `useVoiceRecorder.ts`: it requests microphone access via `getUserMedia`, records with `MediaRecorder` (preferring `audio/webm`), and on stop sends the blob to `transcribeAudio()` in `transcriptionService.ts`, which `POST`s it as `FormData` to `http://127.0.0.1:8000/transcribe`. The returned text is placed into the input field — the user still presses send. The hook surfaces detailed Hebrew error messages for permission-denied, no microphone, microphone busy, and "insecure context" (HTTPS/localhost-only) cases.
- `MicPermissionPrompt.tsx` shows a small inline hint about granting microphone permission.

5.5 Visual design system

- **Accent colors** (`accentStyles.ts`) — a single `ACCENT_STYLES` map keyed by `'red' | 'sky' | 'amber' | 'violet'`, each defining tile background, icon color, ring, badge, progress-bar fill, border, soft background, and button colors. Categories and lessons each carry an `accent` field, so the same components automatically re-theme per category (e.g. medical = red, nature = sky, accidents = amber, war = violet).
- **Tailwind theme** (`tailwind.config.js`) — Heebo as the global sans-serif font (Hebrew-optimized), a custom `emergency` red color, and two custom keyframe animations: `pulse-strong` (1.8s, used on the AI agent button) and `fade-in-up` (0.25s, used for screen transitions and the rotating quote).
- **Icons** — all hand-rolled inline SVG React components in `icons.tsx`, mapped per-category (`categoryIcons.ts`) and per-lesson (`lessonIcons.ts`).

6. Data & Content Layer

6.1 Types — `src/types.ts`

One file defines every shape used across the app: `Category`, `Lesson` (with optional `pages` for the illustrated walkthrough and optional `scenario` for the timed scenario quiz), `LessonPage` / `LessonPageItem` (a header plus a list of label/text/image items — i.e. the "cards"), `ScenarioPage` / `ScenarioOption` (an image + 3 options, exactly one marked `correct: true`), `QuizQuestion`, and chat types (`ChatMessage`, `ChatRole`).

6.2 Content — `src/data/content.ts`

This is the single source of truth for all lesson content. It currently defines:

- **4 categories:** **חירום רפואי** (Medical Emergency, red), **אסונות טבע** (Natural Disasters, sky), **תאונות** (Accidents, amber), **חירום מלחמה** (War Emergency, violet) — only "Medical Emergency" currently has lessons.
- **2 lessons:** **חנק** (Choking — Heimlich maneuver) and **דימום מאסיבי** (Massive Bleeding — tourniquet use), each with a full `pages` walkthrough and 3 `steps` for the checklist/progress badge.
- **CHOKING_PAGES / BLEEDING_PAGES** — each lesson's walkthrough is broken into multiple "header" sections (e.g. "Explanation of the problem", "How to treat it", "Applying a tourniquet", "Recording the critical time"), and each section contains one or more *cards* (an image + a paragraph of text), so every paragraph from the source PDF gets its own visual card.
- **BLEEDING_SCENARIO** — the 3-round timed scenario quiz: (1) a person bleeding from the leg → correct answer is to press hard on the wound with your knee, (2) pressure is being applied → correct answer is to call an ambulance (101), (3) bleeding continues → correct answer is to improvise a tourniquet from a belt and stick.
- **QUIZ_QUESTIONS** — the global quick-quiz questions (one per lesson today), each with a question, 3 options, the correct option id, and Hebrew explanation text for both correct and incorrect answers.

6.3 Where the content came from

All lesson text and many illustrations were **extracted directly from the real reference documents** in `categories/Medical_Conditions/` (`Choking.docx/.pdf` and `Massive_Bleeding_Tourniquet_2.docx/Massive_Bleeding_Tourniquet.pdf`). A `.docx` file is just a ZIP archive — its text lives in `word/document.xml` (inside `<w:t>` tags) and its embedded images are referenced via `r:embed="rIdN"`, cross-referenced against `word/_rels/document.xml.rels` to find the actual image files. This let the team pull out the exact wording and matching diagrams used by **CHOKING_PAGES** and **BLEEDING_PAGES** rather than writing placeholder copy.

6.4 Rotating quotes — `src/data/quotes.ts` + `famous_medical_quotes.json`

A list of 100 famous medical quotes (sourced from `famous_medical_quotes/famous_medical_quotes.json`) is shown one at a time on the home screen by `RotatingQuote.tsx`. It picks a random starting quote, then advances to the next one every 60 seconds with a fade-in-up animation — a small touch of "liveliness" on an otherwise static home screen.

7. AI Backend — ChatBotBackend/main.py

7.1 Startup sequence

1. Loads `knowledge_base.json` — a small curated list of Hebrew first-aid facts/instructions (e.g. choking and bleeding guidance) — and wraps each entry as a LangChain `Document` .
 2. Initializes **Ollama embeddings** (`nomic-embed-text`) and builds an in-memory **Chroma vector store** from those documents.
 3. Initializes a **local LLM** via Ollama (`qwen2.5:0.5b` , temperature 0, capped at 80 tokens) — a very small, fast model suitable for running on a laptop CPU.
 4. Loads a local **faster-whisper "tiny"** model (CPU, int8) for Hebrew speech-to-text.
- If Ollama isn't running/installed, the embedding/vectorstore/LLM initialization is wrapped in a `try/except` that sets them all to `None` — the API still starts, but `/api/chat` responds with a friendly "AI system did not initialize correctly" message instead of crashing.

7.2 API Endpoints

Endpoint	Method	Input	Behavior
<code>/api/chat</code>	POST	JSON { <code>"question": "..."</code> }	Runs a semantic similarity search (<code>k=1</code>) against the Chroma vector store using the user's question. If no good match is found (distance score above a threshold), it replies <code>אינני יודע את התשובה מתוך המידע שסופק</code> ("I don't know the answer from the information provided"). Otherwise it returns the matched knowledge-base fact directly as the answer — i.e. today the system performs retrieval-only RAG (the small LLM is loaded and ready, but the current endpoint returns the retrieved fact verbatim rather than rephrasing it through the LLM, keeping responses fast, deterministic, and grounded).
<code>/transcribe</code>	POST	<code>multipart/form-data</code> with an <code>audio</code> file field	Reads the uploaded audio bytes into memory, runs <code>faster-whisper</code> with <code>language="he"</code> , concatenates the resulting segments, and returns <code>{ "text": "..."} .</code>

7.3 The knowledge base

`knowledge_base.json` currently contains a handful of short, self-contained Hebrew first-aid statements covering choking/Heimlich and bleeding/tourniquet guidance — easy to extend by simply adding more strings to the JSON array, since each one becomes a separately-searchable vector.

7.4 Running the backend

```
# from ChatBotBackend/
pip install -r requirements.txt

# requires Ollama installed & running, with these models pulled:
#   ollama pull nomic-embed-text
#   ollama pull qwen2.5:0.5b

uvicorn main:app --reload --port 8000
```

8. Development Timeline

The project was built collaboratively (2 contributors working in parallel, hence the merge commits) over the course of the hackathon day:

#	Commit	What happened
1	<code>40053ec</code>	First draft of the app — initial React/Vite/Tailwind scaffold and basic screens.
2	<code>e875f0f</code>	Added the first 2 lesson PDFs to seed the demo content.
3	<code>2791cbd</code>	Switched the design to a light theme.
4	<code>02543c2</code>	"With adding AI" — first version of the AI backend integration.
5	<code>89b6856</code>	Merge of the two parallel branches.
6	<code>fe457b1</code>	Added the 100 famous medical quotes (JSON) for the rotating quote widget.
7	<code>9ea7b11</code>	Updated the source files used for the informative lesson data.
8	<code>316eb95</code> / <code>5ab2a88</code>	"With working chatbot" — the AI backend reaches a working end-to-end state; merged in.
9	<code>6bb8f6c</code> / <code>e3d9be9</code> / <code>7825da6</code>	Added working voice recording (mic capture + Whisper transcription), merged in.
10	<code>d5aafa1</code> / <code>1599b77</code>	Major content update: bleeding lesson, lesson/quiz choice flow, scenario quizzes, app icon, rotating quotes — merged.
11	<code>0b7cc60</code> / <code>bf62242</code>	Refined the bleeding scenario quiz (timer, images), lesson timing, and content layout (per-paragraph cards) — merged.
12	<code>76621b4</code>	Final content polish — shortened the choking lesson title to "חנק".

Note: commit timestamps interleave because two people worked on separate branches (frontend/UI vs. AI backend/voice) and merged repeatedly throughout the day.

9. Running the Project Locally

Frontend

```
npm install
npm run dev      # starts Vite on http://localhost:5173 (and your LAN IP, via host: true)
npm run build    # type-checks (tsc) then builds to dist/
npm run preview  # preview the production build
```

AI Backend

```
cd ChatBotBackend
pip install -r requirements.txt
uvicorn main:app --reload --port 8000
```

Dependency note: the AI chat tab requires the FastAPI backend running on port 8000 *and* a local Ollama installation with the `nomic-embed-text` and `qwen2.5:0.5b` models pulled. Without Ollama, the backend still starts and the rest of the app (Learn + Quiz tabs) works fully — only the chat endpoint degrades to a graceful "AI system not initialized" message. Voice transcription (Whisper) does not depend on Ollama and works independently.

HTTPS for mobile testing: microphone access (`getUserMedia`) is blocked by browsers on any non- `localhost` origin unless served over HTTPS. `vite-plugin-mkcert` automatically generates a locally-trusted certificate (including for your LAN IP), so the dev server can be opened securely from a phone on the same Wi-Fi network for live demos.

10. Key Talking Points for Judges

- **Real, sourced content** — lesson text and diagrams come straight from professional first-aid documents, not invented copy, giving the training material genuine credibility.
- **Two complementary learning modes** — calm step-by-step study (illustrated pager) *and* high-pressure timed decision-making (scenario quiz with a 6-second countdown), mirroring how real emergencies actually feel.
- **Fully local AI stack** — RAG pipeline (LangChain + Chroma + Ollama) and speech-to-text (faster-whisper) both run on-device with zero cloud dependency, zero API cost, and full privacy — important for a tool that might handle sensitive medical conversations.
- **Multimodal input** — users can type *or* speak to the AI agent, which matters in a hands-busy emergency.
- **Polished, accessible UI** — native RTL Hebrew support, a cohesive color-coded design system across 4 emergency categories, custom icons, and subtle motion design (pulsing CTA, fade-in transitions, rotating quotes) that make the app feel production-ready rather than a prototype.
- **Extensible architecture** — adding a new emergency category just means adding entries to `content.ts` (and optionally new images); adding new AI knowledge just means appending strings to `knowledge_base.json` — no code changes required for either.